

Karar Yapılarına Giriş: if-else if-else Merdiven Yapısı

Görsel Programlama - Hafta 9



Karar Yapılarına Giriş: if-else if-else Merdiven Yapısı

Bu sunumda, ikiden fazla durumun olduğu senaryolarda kullanılan if-else if-else merdiven yapısını inceleyeceğiz.

Programlamada akış kontrolü, verilen koşullara göre farklı kod bloklarının çalıştırılmasını sağlar.

Temel if-else yapısı yalnızca iki durumu (Doğru/Yanlış) yönetebilirken, else if yapısı daha karmaşık mantıksal yolları yönetmemizi sağlar.

Hedefimiz, bu yapının sentaksını, çalışma prensibini ve gerçek dünya uygulamalarını anlamaktır.



Neden else if Kullanmalıyız?

Kaynakta belirtildiği gibi, else if merdiven yapısı ikiden fazla durumun olduğu senaryolar için gereklidir.

Basit if yapıları (nested if'ler dahil) karmaşık ve okunması zor olabilir.

else if yapısı, koşulların sırayla ve birbirini dışlayacak şekilde kontrol edilmesini sağlar.

Eğer bir koşul doğruysa, diğer koşullar kontrol edilmez ve program akışı merdivenden çıkarak devam eder.



if-else if-else Merdiveninin Sentaksı (C#)

C# dilinde if, else if ve else anahtar kelimeleri kullanılır.

```
if (koşul1)
```

```
{
```

```
    // Koşul 1 doğruysa çalışır
```

```
}
```

```
else if (koşul2)
```

```
{
```

```
    // Koşul 1 yanlış VE Koşul 2 doğruysa çalışır
```

```
}
```

```
else
```

```
{
```

```
    // Hiçbir koşul doğru değilse çalışır
```

```
}
```

Çalışma Prensipleri: Sıralı Kontrol

Program, koşullardan biri doğru olana kadar sırayla kontrol eder.

1. En üstteki if koşulu kontrol edilir.
2. Eğer if doğruysa, ilgili kod bloğu çalışır ve yapının tamamı atlanır.
3. Eğer if yanlışsa, sıradaki else if koşulu kontrol edilir.
4. Bu süreç, doğru bir koşul bulunana kadar devam eder.
5. Tüm if ve else if koşulları yanlışsa, isteğe bağlı olan else bloğu çalıştırılır.

C# Anahtar Kelimeleri Detaylı İnceleme

if: Karar yapısını başlatan zorunlu anahtar kelimedir.

else if: İsteğe bağlıdır ve if'ten sonra istenilen sayıda kullanılabilir. Her biri yeni ve bağımsız bir koşul sunar.

else: İsteğe bağlıdır ve yapının sonuna gelir. Tüm koşulların başarısız olduğu 'varsayılan' durumu yakalamak için kullanılır.

Koşullar, Boole (True/False) değeri döndürmelidir.

Kapsayıcılık ve Mantıksal Hata Kontrolü

if-else if merdiveninde, bir koşulun doğru olması durumunda diğerlerinin kontrol edilmemesi, koşulları tanımlarken dikkatli olmayı gerektirir.

Örnek: Not aralıklarını tanımlarken, aralıkların birbiriyle çakışmadığından emin olunmalıdır. Çakışma olsa bile, merdiven yapısı ilk doğru olanı seçecektir. Örneğin, 90-100 aralığını tanımlarken, bir sonraki else if'in 85-90 aralığını kapsamaması için sıkı sınırlar (örneğin ' < 90 ') kullanılmalıdır.



UI Kontrolleri ile Etkileşim

Kaynak metin, TextBox, Button ve Label gibi form kontrollerinin kullanılacağını belirtir.

TextBox: Kullanıcıdan veri girişi almak için kullanılır (örneğin not veya sayı).

Button: Karar yapısının tetiklenmesi ve kod akışının başlatılması için kullanılır ('Hesapla' gibi).

Label: Karar yapısının sonucunu kullanıcıya görsel olarak sunmak için kullanılır (örneğin harf notu veya sonuç mesajı).



Veri Türü Dönüşümleri (Type Conversion)

Kullanıcı arayüzünden (TextBox) alınan veriler varsayılan olarak string (metin) türündedir.

Karar yapılarını sayısal değerler üzerinde çalıştırmak için bu verilerin int, double veya decimal gibi uygun sayısal türlere dönüştürülmesi gerekir. Kaynak örnekte, notun string'den int'e çevrilmesi gerektiği belirtilmiştir (int not = Convert.ToInt32(txtNot.Text);).

Dönüşüm hatalarını (örneğin, metin girilmesi) ele almak için TryParse veya try-catch blokları kullanmak akademik düzeyde önerilir.



Örnek Proje 1: Notu Harfe Çevirme Sistemi

Bu proje, belirli sayısal aralıklara göre karar veren çoklu koşul yapısı kurmayı amaçlar.

Projenin temel gereksinimi, 0-100 arası not girişini kabul etmek ve buna karşılık gelen harf notunu (AA, BA, BB vb.) belirlemektir.

Bu, else if merdiven yapısının en klasik ve öğretici uygulamalarından biridir.



Proje 1 Arayüz Tasarımı (UI)

Proje arayüzü aşağıdaki bileşenleri içermelidir:

1 adet TextBox (txtNot): Sayısal not girişi için (0-100 arası).

1 adet Button (btnHesapla): 'Hesapla' işlemi için.

1 adet Label (lblSonuc): Harf notu sonucunu göstermek için (örn: 'AA').

Kullanıcı dostu olması için Label'ın başlangıçta boş veya açıklayıcı bir metin içermesi önerilir.

Proje 1 Kod Akışı Başlangıcı

Kod akışı, Button'a tıklandığında (Button Click) başlar.
Öncelikle TextBox'taki metin alınır ve tamsayıya (int) dönüştürülür:
`int not = Convert.ToInt32(txtNot.Text);`
Bu dönüşüm, karşılaştırma operatörlerinin doğru çalışması için kritik öneme sahiptir.

Proje 1: AA Not Aralığı

En yüksek not aralığı ilk olarak kontrol edilmelidir:

if (not ≥ 90 && not ≤ 100):

Bu koşul, notun hem 90'dan büyük veya eşit, hem de 100'den küçük veya eşit olduğunu kontrol eder.

Bu koşul doğruysa, Label'a 'AA' yazdırılır.

Mantıksal VE (&&) operatörü, iki koşulun da aynı anda doğru olmasını zorunlu kılar.

Proje 1: BA Not Aralığı

if koşulu (AA) başarısız olursa, program sıradaki else if'e geçer:

else if (not ≥ 85 && not < 90):

Bu noktada, notun 90'dan küçük olduğunu zaten biliyoruz (çünkü if başarısız oldu).

Bu yüzden, 85'ten büyük veya eşit olması kontrol edilir.

Koşul doğruysa, Label'a 'BA' yazdırılır.

Merdiven yapısı sayesinde, 'not < 90 ' kontrolünü teknik olarak tekrar yazmak zorunda kalmayız, ancak açıkça belirtmek daha güvenli bir programlama pratiğidir.

Proje 1: BB Not Aralığı ve Devamı

İkinci else if örneği:

else if (not >= 80 && not < 85):

Bu koşul, notun 80 ile 84 arasında olduğunu kontrol eder (85'ten küçük olma koşulu önemlidir).

Koşul doğruysa, Label'a 'BB' yazdırılır.

Bu yapı, '...' (Diğer not aralıkları için else if blokları devam eder) şeklinde ilerlemelidir.

Her adımda, üstteki tüm koşulların yanlış olduğu kabul edilerek alt sınıra odaklanılır.

Proje 1: FF Not Aralığı (En Alt Sınır)

En düşük geçerli not aralığı:

else if (not ≥ 0 && not < 50):

Bu, 0 ile 49 arasındaki notları kapsar.

Bu koşul doğruysa, Label'a 'FF' yazdırılır (Başarısız).

Burada 0'dan küçük bir değer girilip girilmediği, aşağıdaki son else bloğu tarafından kontrol edilecektir.

Proje 1: Hata Kontrolü ve Varsayılan Durum

else:

Bu blok, yukarıdaki tüm if ve else if koşullarının (0-100 aralığındaki tüm harf notları) yanlış olduğu anlamına gelir.

Bu genellikle, kullanıcının 0'dan küçük veya 100'den büyük bir değer girdiği anlamına gelir.

Label'a 'Geçersiz Not (0-100 arası girin)' yazdırılır.

Bu, kullanıcıya geri bildirim sağlayan temel bir hata kontrol mekanizmasıdır.

Üniversite düzeyinde, kullanıcı string yerine sayısal olmayan bir metin (örneğin 'A') girdiğinde oluşacak `Convert.ToInt32` hatasını engellemek için `TryParse` kullanılmalıdır.

`int.TryParse(txtNot.Text, out not)` metodu, dönüşüm başarılı olursa `true`, aksi halde `false` döndürür ve programın çökmesini önler.

Merdiven yapısının içine girmeden önce `TryParse` kontrolü yapılmalıdır.

Proje 2: Sayı Sınıflandırma

Proje 2, üç durumu (< 0 , > 0 , $== 0$) kontrol eden temel bir if-else if-else yapısı kurmayı amaçlar.

Bu, else if merdiveninin en yalın ve anlaşılır örneğidir.

Üç olası durumu net bir şekilde ayırmak için idealdir.



Proje 2 Arayüz Tasarımı ve Veri Girişi

Arayüz (UI) yine bir TextBox, bir Button ve bir Label içerir.
Kod akışında, TextBox'tan bir sayı okunur ve uygun bir int veya double veri tipine dönüştürülür.
`int sayi = Convert.ToInt32(txtSayi.Text);` (veya benzeri bir dönüşüm)

Proje 2 Kod Akışı: Pozitif Kontrolü

Sayısal değer alındıktan sonra, en doğal durum (Pozitiflik) kontrol edilir:
if (sayi > 0):
 Label'a 'Sayı Pozitifdir' yazdırılır.
Bu, ilk ve ana koşulumuzdur.

Proje 2 Kod Akışı: Negatif Kontrolü

Eğer sayı 0'dan büyük değilse, Negatiflik kontrol edilir:

else if (sayı < 0):

Label'a 'Sayı Negatiftir' yazdırılır.

Bu else if bloğu, sayının kesinlikle pozitif olmadığını garanti ederken, negatif olup olmadığını kontrol eder.

Proje 2 Kod Akışı: Sıfır Durumu

Eğer sayı ne pozitif ($\text{sayi} > 0$) ne de negatif ($\text{sayi} < 0$) ise, kalan tek mantıksal seçenek nedir?

else:

Label'a 'Sayı Sıfırdır' yazdırılır.

Bu else bloğu, ' $\text{sayi} == 0$ ' koşulunu dolaylı olarak temsil eder ve yapıyı tamamlar.

if-else if vs. iç içe if (Nested if)

İç içe if yapıları (Nested if), koşullar hiyerarşik veya bağımlı olduğunda kullanılır.

else if merdiveni, bağımsız ama birbirini dışlayan koşulları (mutually exclusive) kontrol etmek için daha uygundur.

else if: Daha düz, daha hızlı çalışır (çünkü ilk doğru koşuldan sonra kontrol durur) ve okunabilirliği yüksektir.

Nested if: Derin iç içe yapılar 'spagetti koduna' yol açabilir ve bakımı zordur.

if-else if Merdiveninin Performansı

else if merdiveninde kontrol, koşullardan biri doğru olana kadar yapılır. Bu, doğru koşulun listenin başında olması durumunda programın çok hızlı çalışacağı anlamına gelir. Performans optimizasyonu için, en sık beklenen veya en kısıtlayıcı koşullar merdivenin en üstüne yerleştirilmelidir. Eğer ilk koşul doğruysa, CPU, kalan else if bloklarını tamamen atlar.

Kapsayıcılık Hatası (Overlap Error)

Kapsayıcılık hatası, iki koşulun aynı anda True olabileceği durumlarda ortaya çıkar.

Örnek (Hata): `if (not > 80)` ve `else if (not > 90)`.

Eğer not 95 ise, ilk `if (not > 80)` doğru olacak ve program alt bloğa inmeyecektir.

Bu nedenle, harf notu örneğinde olduğu gibi, daha kısıtlayıcı koşullar (90-100) önce gelmelidir.

Mantıksal Operatörlerin Rolü

Çoklu koşul yapıları, karmaşık karar mekanizmaları oluşturmak için mantıksal operatörleri kullanır.

Proje 1'de olduğu gibi ($\text{not } \geq 90 \ \&\& \ \text{not } \leq 100$), $\&\&$ operatörü, bir aralığı tanımlamak için zorunludur.

$\parallel$ (VEYA) operatörü, koşullardan herhangi birinin doğru olmasının yeterli olduğu durumlarda kullanılır.

Bu operatörlerin doğru kullanımı, else if merdiveninin etkinliğini artırır.

Karşılaştırma Operatörleri

Karar yapılarında kullanılan temel karşılaştırma operatörleri şunlardır:
> (Büyüktür), < (Küçüktür), >= (Büyük veya Eşittir), <= (Küçük veya Eşittir).
== (Eşittir), != (Eşit Değildir).

Proje 2'de 'sayi > 0' ve 'sayi < 0' kullanılmıştır.

Karar yapılarında tek eşittir (=) atama operatörünün yanlışlıkla kullanılmaması hayati önem taşır.



switch Deyimi: Alternatif Yaklaşım

Çok sayıda eşitlik kontrolü yapılması gerektiğinde (örneğin gün adları, sabit durum kodları), switch deyimi else if merdivenine daha temiz bir alternatif sunar.

switch: Belirli bir değişkenin tam değerini karşılaştırır.

else if: Hem tam değer hem de aralıklar ($>$, $<$, $>=$) için uygundur.

Not dönüştürme gibi aralık bazlı işlemler için else if daha uygundur, ancak kesin değer kontrolleri için switch tercih edilebilir.



switch vs. else if: Okunabilirlik ve Bakım

switch deyimi, çok sayıda case içerdiğinde bile genellikle daha iyi okunabilirlik sunar.

Ancak, switch yalnızca tamsayı türleri, stringler veya enum'lar gibi sınırlı türler üzerinde çalışır.

else if merdiveni, herhangi bir Boole ifadesini desteklediği için daha esnektir (örneğin, karmaşık mantıksal operatörler içeren koşullar).



Else Bloğunun Önemi

Else bloğu isteğe bağlıdır, ancak profesyonel programlamada sıklıkla kritik öneme sahiptir.

Else, beklenen koşullar dışında kalan tüm senaryoları yakalar.

Bu, hem hata yakalama (Proje 1'deki 'Geçersiz Not' gibi) hem de varsayılan bir işlemi (Proje 2'deki 'Sıfır' durumu gibi) gerçekleştirmek için kullanılır.

Else'in varlığı, programın hiçbir girdi/durum için tanımsız kalmamasını garanti eder.



Dinamik else if Yapıları

Akademik uygulamalarda, else if koşulları bazen sabit değerler yerine, başka değişkenlerden veya veritabanından gelen değerlere bağlı olabilir. Örneğin, not sınırlarının (90, 85, 80...) dinamik olarak konfigürasyon dosyasından okunması. Bu, kodun daha esnek olmasını ve farklı organizasyonların notlandırma sistemlerine uyum sağlamasını sağlar.



Kısa Devre Değerlendirmesi (Short-Circuit Evaluation)

C# ve diğer birçok dilde && ve || operatörleri kısa devre değerlendirme yapar.

&& (AND) için: İlk koşul yanlışsa, ikinci koşul kontrol edilmez (çünkü sonuç kesinlikle False olacaktır).

|| (OR) için: İlk koşul doğruysa, ikinci koşul kontrol edilmez (çünkü sonuç kesinlikle True olacaktır).

Bu, else if yapısında olduğu gibi, gereksiz hesaplamaları önleyerek performansı artırır.



Kod Tekrarından Kaçınma (DRY Prensibi)

else if merdiveni, koşullu bloklarda tekrar eden kodlar içeriyorsa, bu kod parçaları bir metoda taşınmalıdır.

Örneğin, her not aralığında Label'a yazma işlemi tekrarlanmaktadır. Bu temel bir tekrar olsa da, karmaşık uygulamalarda aynı veri manipülasyonunun birden fazla koşulda yapılmasından kaçınılmalıdır.

Bu, kodun bakımını kolaylaştırır ve hata potansiyelini azaltır.

Sıralama Hatası (Ordering Error)

else if merdiveninde koşulların sırası kritik öneme sahiptir.

Genel kural: En spesifik (en dar) koşullar en başta, en genel (en geniş) koşullar en sonda olmalıdır.

Not örneğinde, 90-100 (en dar pozitif aralık) en başta, 0-100 dışında kalan (en geniş hata aralığı) en sonda yer almalıdır.

Yanlış sıralama, doğru koşulların hiçbir zaman test edilmemesine neden olabilir.



Kullanım Alanları: Tıbbi Tanı Sistemleri

else if yapısı, ardışık kararların verildiği sistemlerde yaygındır. Tıbbi sistemlerde, belirtilere dayalı olarak bir dizi potansiyel tanıyı kontrol etmek için kullanılabilir (örneğin, eğer A belirtisi varsa X'i kontrol et, aksi takdirde Y'yi kontrol et). Bu, bir karar ağacının kodlanmış halidir.

Kullanım Alanları: Fiyatlandırma ve İndirim Hesaplamaları

E-ticaret sistemlerinde, müşterinin sepet tutarına veya üyelik seviyesine göre kademeli indirimler uygulamak için else if merdiveni kullanılır.

if (sepet > 1000 TL): %20 indirim

else if (sepet > 500 TL): %10 indirim

else: indirim yok.

Bu, Proje 1'deki not aralığı mantığına benzer bir yapı sunar.

Kullanım Alanları: Oyun Programlama ve Durum Yönetimi

Oyunlarda karakterin sağlık durumu, mühimmat seviyesi veya düşmanın yapay zeka davranışları gibi durumları yönetmek için kullanılır.

if (Sağlık < 10): Kritik hasar animasyonu

else if (Sağlık < 50): Orta hasar animasyonu

else: Normal durum.

Bu, oyun akışını yöneten temel mekanizmalardan biridir.

Veri Tipi Sınırlamaları ve Kontrol

else if merdiveni sadece tam sayılarla (int) çalışmaz. Kayar noktalı sayılar (double, float) veya hatta string veriler üzerinde de karşılaştırmalar yapılabilir. Örnek: if (isim == 'Ahmet') veya else if (ortalama > 3.5). Ancak string karşılaştırmalarında büyük/küçük harf duyarlılığı gibi ek konular dikkate alınmalıdır.



Boolean Karşılaştırmaları (Flags)

else if yapısı, doğrudan Boole değişkenlerini (True/False bayrakları) kontrol etmek için de kullanılabilir.

Örneğin: `if (isLoggedIn) { ... } else if (isGuest) { ... }.`

Bu tür kullanımlar, programın farklı kullanıcı yetkilerine veya oturum durumlarına göre tepki vermesini sağlar.

Tersine Mühendislik: if/else Yapısını Merdivene Dönüştürme

Derin iç içe if yapıları (Nested ifs), genellikle else if merdivenine dönüştürülerek basitleştirilebilir.

Kod Yeniden Yapılandırma (Refactoring) sürecinde, karmaşık koşulların düzleştirilmesi okunabilirliği artırır ve bakımı kolaylaştırır.

Ancak, koşullar gerçekten hiyerarşik bağımlılığa sahipse, nested if korunmalıdır.



Merdiven Yapısında Süslü Parantez Kullanımı

C#'ta, if veya else if bloklarından sonra yalnızca tek bir ifade gelecekte süslü parantezler ({}) zorunlu değildir.

Ancak, akademik düzeyde ve kurumsal projelerde, okunabilirliği ve hataları önlemek için her zaman süslü parantez kullanılması kesinlikle tavsiye edilir. Süslü parantez olmadan kodun kapsamının yanlış anlaşılması yaygın bir hatadır.



Null Kontrolleri (NPE Önleme)

else if merdiveni, bir değişkenin Null (boş) olup olmadığını kontrol etmek için de kullanılmalıdır (özellikle kullanıcı girdileri ve nesneler söz konusuysa).

```
if (nesne != null) { ... } else { // Hata işle }
```

C# 8.0 ve sonrası null atıf uyarıları (nullable reference types) bu kontrolleri daha güvenli hale getirmiştir.

Mantıksal Tutarlılık Kontrolü

Etkili bir else if merdiveni, ele alınması gereken tüm olası durumları (input domain) kapsamalıdır.

Örneğin, not uygulaması 0-100 aralığını ve bu aralığın dışındaki hatalı girişleri de ele almalıdır.

Sıfırın pozitif veya negatif sayılardan ayrı olarak ele alınması, üçlü kontrolün zorunlu sonucudur.



Koşullu Operatör (Ternary Operator)

İki durumlu basit if/else yapıları için (ancak else if merdiveni için değil), C#'ta `?:` (Ternary Operator) kullanılabilir.

Örnek: `string sonuc = (sayi > 0) ? 'Pozitif' : 'Negatif veya Sıfır';`

Bu operatör, kodu kısaltır ancak karmaşık else if yapılarının yerini alamaz.

İlkel Veri Tiplerinin Kontrolü

Kaynakta kullanılan not ve sayı değişkenleri int (tamsayı) türündedir. int, çoğu basit sayma ve puanlama sistemi için idealdir. Ancak, finansal hesaplamalar gibi hassasiyet gerektiren durumlarda decimal veya double türlerinin kullanılması, karşılaştırmaların doğruluğu açısından önemlidir.



Form Kontrolü: TextBox Kullanımı Detayları

TextBox kontrolünün bazı önemli özellikleri vardır:

Text Özelliği: Kullanıcının girdiği string değeri içerir.

MaxLength: Girişin uzunluğunu sınırlayarak hata olasılığını azaltabilir.

TextChanged Olayı: Kullanıcı her karakter girdiğinde tetiklenen olay (genellikle sadece Button Click olayı tercih edilir).



Form Kontrolü: Button ve Olay Yönetimi

Button kontrolü, program akışını başlatan bir olay tetikleyici görevi görür. Click olayı (Button_Click), tüm karar yapısının çalıştırıldığı yerdir. Olay odaklı programlamada, kodun yürütülmesi kullanıcı eylemlerine bağlıdır.

Form Kontrolü: Label ile Geri Bildirim

Label, kullanıcının girdiği verilerin işlenmesi sonucunda elde edilen çıktıyı (harf notu, pozitif/negatif bilgisi) gösterir.
Label kontrolünün metin içeriği (Text özelliği), else if blokları içinde dinamik olarak değiştirilir.



Kod Okunabilirliği ve Yorumlar

Karmaşık else if merdivenlerinde, hangi aralığın hangi sonucu verdiğini açıklayan yorumlar eklemek çok önemlidir.

Üniversite projelerinde, her else if bloğunun üstüne ilgili koşulu (örneğin // BA aralığı) belirten yorumlar yazılması şiddetle tavsiye edilir.

Yorumlar kodun mantıksal akışını takip etmeyi kolaylaştırır.

Çoklu Koşullu İfadelerin Karmaşıklığı

Çok uzun else if merdivenleri, yazılım mühendisliğinde 'yüksek döngüsel karmaşıklık' yaratır.

Yüksek karmaşıklık, kodun test edilmesini, anlaşılmasını ve sürdürülmesini zorlaştırır.

Eğer bir merdiven yapısı çok uzarsa (örneğin 10'dan fazla else if), switch deyimi veya strateji deseni gibi alternatif çözümler düşünülmelidir.

Test Edilebilirlik (Testability)

else if merdivenini test ederken, her bir koşulun sınır değerleri (boundary conditions) kontrol edilmelidir.

Proje 1'de, 90, 89, 85, 84, 50, 49, 0 ve 101 gibi değerler (aralıkların uç noktaları ve hemen dışı) test edilmelidir.

Sınır koşullarının test edilmesi, merdiven yapısındaki mantıksal hataları (off-by-one errors) ortaya çıkarır.

Negatif Koşulların Kullanımı

if-else if yapılarında bazen bir durumun olmamasını kontrol etmek gerekebilir. Mantıksal deęilleme (!) operatörü bu durumlarda kullanılır (Örn: if (!(sayi < 0)). Ancak, else if merdiveni zaten 'üstteki koşulun yanlış olduęu' varsayımıyla çalıştığı için, negatif koşullar genellikle else if yapısının içine dahil edilmez.

If/Else If Merdiveninde Veri Erişim Kapsamı

Merdiven yapısındaki her bir if, else if veya else bloğu kendi kapsamını ({}) oluşturur.

Bir blok içinde tanımlanan yerel değişkenler, diğer bloklardan erişilemez. Ancak, TextBox'tan alınan 'not' veya 'sayi' değişkenleri, genellikle merdivenin hemen dışında tanımlandığı için tüm if-else if blokları tarafından erişilebilir durumda olmalıdır.



Gelecek Konu: Dizilerle Karar Verme

Büyük ölçekli projelerde, binlerce koşulu manuel else if olarak yazmak yerine, bu koşullar dizilerde veya listelerde saklanabilir. Program, bu dizileri döngüye alarak koşulları dinamik olarak değerlendirebilir. Bu, else if merdiveninin ölçeklenebilirliğini artırmak için kullanılan gelişmiş bir yaklaşımdır.



Özet: else if Merdiveninin Avantajları

1. ****Verimlilik:**** Koşullardan biri doğru olduğunda kontrol hemen durur.
2. ****Netlik:**** Birden fazla bağımsız durumu net bir şekilde ayırır.
3. ****Kapsayıcılık:**** Else bloğu sayesinde tüm olası durumlar ele alınabilir.
4. ****Esneklik:**** Aralık bazlı karşılaştırmalar (Proje 1) için idealdir.



Ozet: else if Merdiveninin Dezavantajları ve Dikkat Edilmesi Gerekenler

1. ****Sıra Hassasiyeti:**** Koşulların sırası yanlışsa hatalı sonuçlar alınabilir.
2. ****Gereksiz Tekrar:**** Çok uzun yapılar döngüsel karmaşıklığı artırır.
3. ****Eşitlik Kontrolü:**** Sadece eşitlik kontrolü yapılyorsa (`==`), switch daha temiz bir alternatiftir.
4. ****Hata Kontrolü:**** Kullanıcı girdisi alınırken TryParse veya try-catch kullanılması zorunludur.



Uygulama Alanları Tekrarı

Notlandırma Sistemleri (Aralık bazlı).
Sayı Sınıflandırması (Pozitif/Negatif/Sıfır).
Hız Limitleri ve Ceza Hesaplamaları.
Kullanıcı Rolü ve İzin Kontrolleri (karmaşık yetki seviyeleri).
Veri Validasyonu (Girdinin belirli standartlara uyup uymadığı).

Proje 1: Tüm Kodun Yapısal Görünümü (Harf Notu)

```
int not = Convert.ToInt32(txtNot.Text);  
if (not >= 90 && not <= 100) { lblSonuc.Text = 'AA'; }  
else if (not >= 85 && not < 90) { lblSonuc.Text = 'BA'; }  
else if (not >= 80 && not < 85) { lblSonuc.Text = 'BB'; }  
// ... Diğer not aralıkları devam eder.  
else if (not >= 0 && not < 50) { lblSonuc.Text = 'FF'; }  
else { lblSonuc.Text = 'Geçersiz Not (0-100 arası girin)'; }
```

Proje 2: Tüm Kodun Yapısal Görünümü (Sayı Kontrolü)

```
int sayi = Convert.ToInt32(txtSayi.Text);  
if (sayi > 0) { lblSonuc.Text = 'Sayı Pozitiftir'; }  
else if (sayi < 0) { lblSonuc.Text = 'Sayı Negatiftir'; }  
else { lblSonuc.Text = 'Sayı Sıfırdır'; }  
Bu yapı, üç olası durumu minimum kod tekrarıyla çözer.
```

Daha Fazla Slayt Gerekliyse: Proje 1 Detayları

Not aralıklarını detaylandıralım (90 AA, 85 BA, 80 BB, 75 CB, 70 CC, 65 DC, 60 DD, 50 FD, 0 FF):

```
else if (not >= 75 && not < 80) { lblSonuc.Text = 'CB'; }  
else if (not >= 70 && not < 75) { lblSonuc.Text = 'CC'; }  
else if (not >= 65 && not < 70) { lblSonuc.Text = 'DC'; }  
else if (not >= 60 && not < 65) { lblSonuc.Text = 'DD'; }  
else if (not >= 50 && not < 60) { lblSonuc.Text = 'FD'; }
```



Gelişmiş Konu: Kodlama Standartları (StyleCop)

Üniversite sonrası profesyonel yazılım geliştirmede, else if merdivenleri gibi yapıların belirli kodlama standartlarına (örneğin StyleCop veya Roslyn Analizörleri) uyması beklenir.

Bu standartlar, boşluk, süslü parantez yerleşimi ve yorumlama kurallarını zorunlu kılar, böylece farklı geliştiriciler aynı stilde kod yazar.



İstisnalar ve Özel Durumlar

else if yapısının kendisi mantıksal hataları yakalar, ancak programın çalışma zamanı hatalarını (Runtime Exceptions) yakalamaz.

Convert.ToInt32 sırasında metin girilmesi, FormatException fırlatır.

Bu tür hataları yakalamak için try-catch blokları else if yapısını çevrelemelidir:

```
try
{
    // Convert ve else if merdiveni burada
}
catch (FormatException e)
{
    // Kullanıcıyı bilgilendir
}
```

Genel Tekrarlama: Karar Yapılarının Amacı

Karar yapıları (if, else if, else) programın sadece yukarıdan aşağıya doğru düz bir çizgide çalışmasını engeller.

Bunun yerine, programın dış etkenlere (kullanıcı girdisi, sistem durumu, dosya içeriği) göre farklı yollara sapmasını sağlar.

Bu, tüm modern yazılımın temelini oluşturan esneklik ve zekayı sağlar.



Boolean İfadelerin Karmaşıklığı

Koşulları (özellikle else if koşullarını) yazarken karmaşıklaşan Boole ifadelerini basitleştirmek için De Morgan yasaları kullanılabilir.

Örn: $\neg(A \ \&\& \ B)$ eşittir $(\neg A \ || \ \neg B)$.

Bu, koşulun mantığını daha net hale getirmeye yardımcı olur ve programlama hatalarını azaltır.



Sonsuz else if Zinciri Sorunu

Teorik olarak else if merdiveninin uzunluğunda bir sınır yoktur, ancak pratik yazılım mühendisliğinde çok uzun zincirler kötü tasarım olarak kabul edilir. Eğer 15-20'den fazla else if bloğu gerekiyorsa, bu genellikle bir veri tablosu araması, switch deyimi veya polimorfizm (nesne tabanlı programlama prensipleri) kullanılması gerektiğinin bir işaretidir.

Koşulların Bağımlılığı: Nested if Yeniden Ziyaret

Eğer ikinci bir koşul, yalnızca ilk koşul doğruysa mantıklıysa, bu durumda nested if kullanmak else if kullanmaktan daha uygundur.

Örn: `if (Kullanıcı_Giriş_Yaptı) { if (Kullanıcı_Admin) { ... } }`.

Burada 'Admin' olma durumu, 'Giriş Yapmış Olma' durumuna bağlıdır.

Kapsam: else if Dışında Tanımlanan Değişkenler

Proje 1'de 'not' değişkeninin if-else if merdiveninden önce tanımlanması, her blok içinde kullanılmasına izin verir.

Eğer 'not' değişkeni sadece if bloğu içinde tanımlanmış olsaydı (ki bu mantıksız olurdu), diğer else if blokları buna erişemezdi.



Örnek: Proje 1'de Sınır Koşulu Testi

Test Durumu 1: Not = 90. Sonuç: AA (Sınır doğru olmalı).

Test Durumu 2: Not = 89. Sonuç: BA (Sınır doğru olmalı).

Test Durumu 3: Not = 49. Sonuç: FF.

Test Durumu 4: Not = -5. Sonuç: Geçersiz Not.

Bu testler, else if aralıklarının doğru tanımlandığını kanıtlar.

Örnek: Proje 2'de Tüm Olasılıklar

Test Durumu 1: Sayı = 10. Sonuç: Pozitiftir.

Test Durumu 2: Sayı = -1. Sonuç: Negatiftir.

Test Durumu 3: Sayı = 0. Sonuç: Sıfırdır.

Else bloğunun sıfırı yakalaması, if-else if yapısının basitliğini gösterir.

else if Merdivenleri ve Fonksiyonel Programlama

Modern programlama paradigmlarında (örneğin fonksiyonel programlama), uzun if-else if merdivenleri yerine desen eşleştirme (pattern matching) gibi daha deklaratif yöntemler tercih edilir.

C# 8.0 ve üzeri versiyonlar, switch ifadeleri ve switch desenleri ile else if merdivenine daha temiz alternatifler sunmaya başlamıştır.



Mantıksal VE (&&) Kullanımı Hakkında Uyarı

Eğer notları 100'den geriye doğru kontrol ediyorsak, else if (not ≥ 85 && not < 90) koşulundaki 'not < 90 ' kısmı teknik olarak gereksizdir. Çünkü ilk if (not ≥ 90) başarısız olduysa, notun zaten 90'dan küçük olduğunu biliyoruz. Ancak, kodun bağımsız okunabilirliğini sağlamak adına bu sınırları açıkça belirtmek yaygın ve güvenli bir uygulamadır.

else if Merdivenleri ve Debugging

if-else if merdivenindeki hataları ayıklamak (debugging), program akışını adım adım takip etmeyi gerektirir.

Program akışının nerede merdivenden çıktığını ve hangi koşulun ilk olarak 'True' döndüğünü anlamak kritik öneme sahiptir.

Visual Studio gibi IDE'lerde Breakpointler (kesme noktaları) bu sürecin vazgeçilmezidir.



Özet ve Temel Terimler

if: Koşullu yapıyı başlatır.

else if: Sıralı alternatif koşulları kontrol eder.

else: Tüm koşullar yanlışsa çalışır.

TextBox, Button, Label: Kullanılan UI bileşenleri.

Convert.ToInt32: String girdiyi sayıya dönüştürür.

&&: Mantıksal VE operatörü.

Kapanış: if-else if-else Yapısına Hakimiyet

if-else if-else merdiveni, bir programcının araç kutusundaki en temel ve güçlü akış kontrol mekanizmasıdır.

Bu yapıyı ve onun sınır koşullarını doğru anlamak, daha karmaşık switch ve desen eşleştirme konularına geçiş için sağlam bir temel oluşturur.



Karar Yapılarına Giriş: if-else if-else Merdiven Yapısı

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

